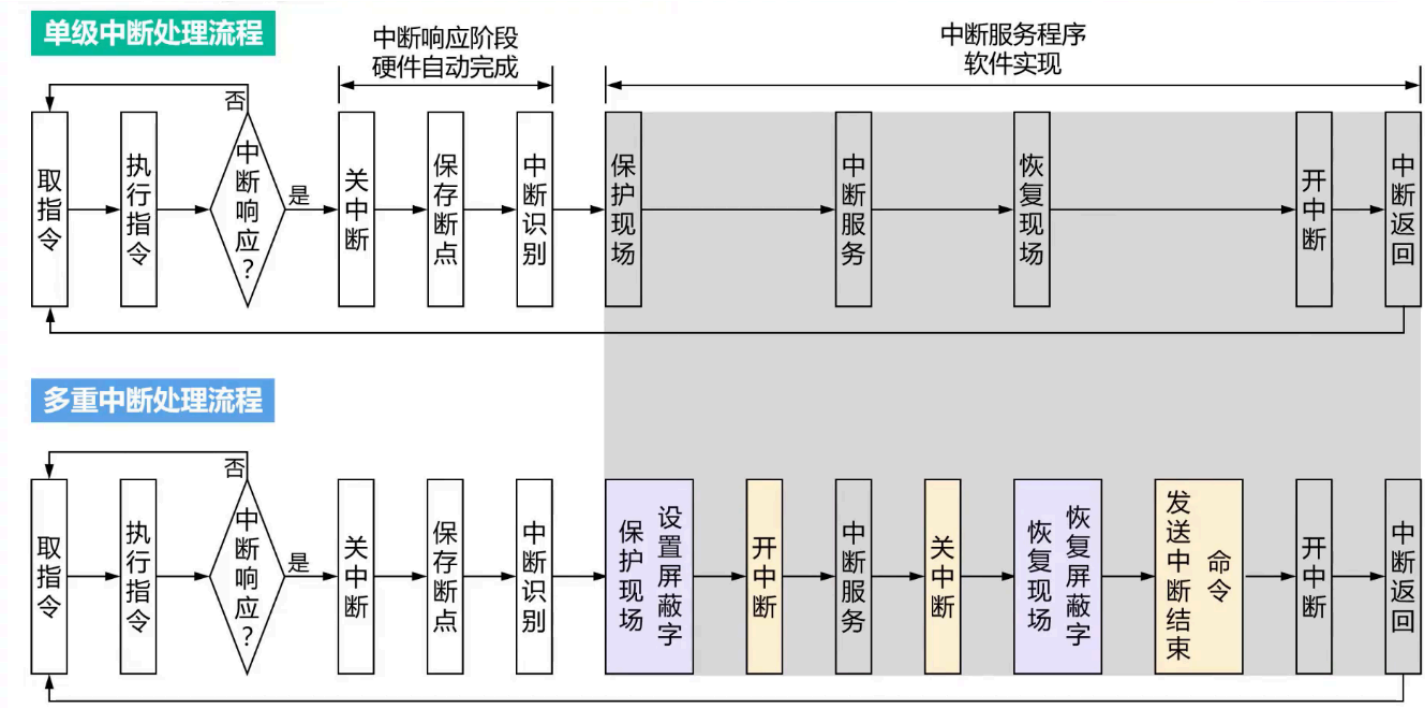
8.11 多重中断的处理流程

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.11）｜重要度 ★★★★｜常见考法：选择题（屏蔽字、两次开关中断的位置、EOI 时机），高频考点

一句话核心：多重中断 = 单级中断的骨架 + 屏蔽字机制 + 两次开关中断；处理优先级由屏蔽字决定

一、多重中断流程图



这张对比图的精髓就一句话：多重中断 = 单级中断的骨架 + 屏蔽字机制 + 两次开关中断。上下两图灰色重叠区是共同的（中断响应三连硬件完成），下半图新出现的三类框（紫 = 屏蔽字、黄 = 开关中断与 EOI）才是考点所在

1.1 与单级中断的差异

新增步骤	位置	作用
设置屏蔽字	保护现场之后、开中断之前	定制“允许谁打断我”，开中断前必须先设好
第一次开中断	设置屏蔽字之后、中断服务之前	放行嵌套：让更高优先级请求能进来
第二次关中断	中断服务之后、恢复现场之前	锁门护场：保证恢复现场不被打断
发送中断结束命令（EOI）	恢复现场之后	通知 8259A 清 ISR 位，放行后续同级/低级请求

1.2 几次中断的开关

- **第一次开中断**：只放行处理优先级更高的请求（低级和同级被屏蔽字挡住，仍登记在 8259A 的 IRR 里等着）
- **第二次关中断**：保护“恢复现场 + 恢复屏蔽字”这段操作的原子性——现场恢复到一半被改写就全乱了

- **最后开中断**：返回主程序前恢复 CPU 的响应能力。注意此时断点 PSW 的弹出（IRET）会连同 IF 位一起恢复，这里体现为流程图上的显式开中断

1.3 EOI 命令的位置

“发送中断结束命令”就是 CPU 向 8259A 发 EOI，清 ISR 对应位。放这里的逻辑：现场和屏蔽字都恢复了，才轮到告诉 8259A“这个中断我彻底办完了”。如果漏发，ISR 位一直是 1，同级和低级中断从此进不来（PR 判优时压不过正在服务的那个）

1.4 完整流程

1. 中断隐指令（硬件）：关中断、保存断点（PC + PSW）、中断识别
2. 保护现场：压通用寄存器
3. 设置屏蔽字：软件写 IMR，定制允许谁打断
4. 开中断：允许更高处理优先级的中断嵌套
5. 中断服务：执行具体处理
6. 关中断：保护恢复现场
7. 恢复现场 + 恢复屏蔽字：弹回通用寄存器，恢复原来的 IMR
8. 发送 EOI：清 8259A 的 ISR 对应位
9. 开中断：恢复 CPU 响应能力
10. 中断返回：IRET 弹回 PC + PSW

1.5 易错点清单

- **易错 1：认为开中断后同级/低级也能嵌套进来。** 错。能插进来的只有处理优先级更高的；同级和低级被屏蔽字挡住，只是继续在 IRR 里挂起，不会丢失
- **易错 2：混淆两种优先级。** “同时到达谁先被响应”看响应优先级（硬件排队器）；“正在执行时谁能打断我”看处理优先级（屏蔽字）。题目给两个次序让你填屏蔽字，考的全是处理优先级
- **易错 3：屏蔽字设置与开中断的顺序颠倒。** 必须先设屏蔽字再开中断；恢复时先恢复现场和屏蔽字再开中断。任何“开中断裸奔在防护未设好时”的选项都是错的
- **易错 4：认为屏蔽字恢复可以省略。** 屏蔽字若不恢复，下一层嵌套返回后，本层设的屏蔽还在生效，会错误地持续屏蔽其他中断源
- **易错 5：把“设置屏蔽字”当成隐指令。** 屏蔽字是服务程序里用特权指令设置的（软件），不是硬件自动完成的。隐指令三件事（关中断/存断点/中断识别）在上下两图中完全相同，灰色区重叠正是这个含义
- **易错 6：认为多重中断每一层的流程不一样。** 每一层嵌套进来的 ISR 都执行同一套模板（设屏蔽字 → 开 → 服务 → 关 → 恢复 → EOI → 开 → 返回），嵌套只是把这套模板叠罗汉
- **易错 7：漏掉“中断请求不丢失”的理解。** 被屏蔽的请求不会消失，它在 8259A 的 IRR 里保持挂起，等屏蔽解除（或 ISR 清零）后 PR 重新判优——这也解释了为什么低级中断只是“延迟处理”而非“丢弃”

一句话收束：多重中断的流程变化全在软件段——进场先设屏蔽字再开门，出门先锁门再恢复、补发 EOI 再走人；屏蔽字让“谁打断谁”从硬件铁律变成软件可调，这就是响应优先级与处理优先级的分野

二、多重中断服务程序与子函数的对比

下表展示了中断服务程序（一种由事件驱动的、由硬件密切管理的特殊程序）与子函数（一种由软件控制的、逻辑复用的常规程序）在核心机制上的本质区别。

对比维度	中断服务程序	子函数
调用方式	由硬件或软件中断事件随机触发，无明确调用者。	在程序中通过调用指令显示调用，调用时机由代码逻辑决定。
PC的保存与修改	● 由硬件中断隐指令自动完成。 ● 入口地址由中断识别机制（如中断向量表）提供。	● 由专用调用指令（如x86的call、MIPS的jal）完成。 ● 入口地址由调用指令本身编码给出。
保护现场的内容	● 更多的寄存器。 ● 由于没有“调用者”，除了自身要修改的寄存器，还需额外保存ABI（Application Binary Interface）中约定的由“调用者保存的寄存器”，也就是所有会被中断服务程序改写的寄存器。	● 部分寄存器。 ● 只需按约定保存自身会修改的“被调用者保存寄存器”。
返回主程序的方式	● 中断服务程序返回与函数调用返回使用的指令不同。 ➢ 对于中断服务程序的返回，x86使用iret，MIPS使用eret。 ➢ 对于函数调用的返回，x86使用ret，MIPS使用jr $ra。	

对比维度	中断服务程序（ISR）	子函数
调用方式	由硬件或中断事件随机触发，无明确调用者	在程序中通过调用指令显式调用，调用时机由代码决定
PC 的保存与修改	由中断隐指令自动保存 PC + PSW；入口地址由中断向量表提供	由调用指令保存返回地址；入口地址由调用指令给出
保护现场的内容	需要保存更多寄存器；由于没有“调用者”，除了自身要修改的寄存器，还要额外保存可能被破坏的寄存器	按调用约定只保存必要的寄存器；由调用者保存的寄存器由调用者负责
返回主程序的方式	用中断返回指令（如 x86 的 IRET、MIPS 的 eret）恢复 PC + PSW	用普通返回指令（如 x86 的 RET、MIPS 的 jr $ra）恢复返回地址
与主程序的关系	异步，可在任意指令边界被打断，可嵌套	同步，由代码显式控制，可嵌套但时机确定

- 关键差异：ISR 的“调用者”是被中断的程序，但它并没有主动调用 ISR，所以 ISR 必须自己保护可能被破坏的现场
- ISR 的返回要恢复 PC + PSW；子函数返回只恢复返回地址（以及调用约定规定的寄存器）
- 这也解释了为什么 ISR 的开销通常大于普通函数调用

三、易错辨析

易混点	核心区分
响应优先级 vs 处理优先级	响应优先级决定同时到达谁先响应，硬件固定；处理优先级决定执行中谁能打断谁，由屏蔽字动态调整
第一次开中断 vs 最后开中断	第一次开中断在设屏蔽字之后，放行嵌套；最后开中断在 EOI 之后，恢复主程序响应能力
第二次关中断 vs 隐指令关中断	第二次关中断在服务之后，保护恢复现场；隐指令关中断在响应之初，保护保存断点
EOI 的位置	在恢复现场和屏蔽字之后、最后开中断之前；漏发会导致同级/低级中断无法响应
屏蔽字 vs 关中断	屏蔽字按位屏蔽某个中断源；关中断（IF=0）屏蔽所有可屏蔽中断
ISR vs 子函数	ISR 由硬件随机触发、硬件保存 PC+PSW、返回用 IRET；子函数由指令显式调用、调用指令保存返回地址、返回用 RET
嵌套 vs 中断请求不丢失	被屏蔽的请求仍挂在 IRR，不会丢失；嵌套是当前 ISR 被更高处理优先级打断

四、考法透视

常见题型

- 给多重中断流程图，判断“设置屏蔽字、开中断、关中断、EOI”的位置
- 给响应优先级和屏蔽字，判断嵌套顺序/CPU 轨迹
- 判断某请求在何时被屏蔽、何时被响应、是否会丢失
- 比较 ISR 与子函数的调用/返回/现场保护
- 结合 8.9 的 8259A 流程，判断 EOI 与 ISR 位的关系

常见设错点

1. 认为开中断后同级/低级也能嵌套
2. 把响应优先级当成处理优先级
3. 先开中断再设屏蔽字
4. 恢复时忘记恢复屏蔽字
5. 把设置屏蔽字当成硬件隐指令
6. 认为每一层嵌套的 ISR 流程不同
7. 认为被屏蔽的请求会丢失
8. 认为 ISR 返回用普通 RET

五、知识关联

- 前置：8.8 中断优先级和中断屏蔽、8.9 中断请求、8.10 单级中断处理流程
- 后续：8.12 DMA控制方式（DMA 结束也通过中断善后）
- OS 视角：中断嵌套、临界区、关中断/开中断
- 全章导航：8.0 导航与术语

六、费曼自测

1. 多重中断比单级中断多了哪三个关键步骤 → 设置屏蔽字、第一次开中断（放行嵌套）、第二次关中断（保护恢复现场），以及 EOI
2. 第一次开中断和最后开中断的作用分别是什么 → 第一次开中断放行更高处理优先级请求；最后开中断恢复主程序的响应能力
3. 为什么恢复现场前必须关中断 → 防止恢复现场到一半被新中断打断，寄存器现场被破坏
4. EOI 放在哪里，漏发会怎样 → 放在恢复现场和屏蔽字之后、最后开中断之前；漏发会导致 ISR 位一直为 1，同级/低级中断无法响应
5. ISR 和子函数在调用与返回上有什么区别 → ISR 由硬件随机触发，硬件保存 PC+PSW，返回用 IRET；子函数由调用指令显式调用，调用指令保存返回地址，返回用 RET